```
15      refute_enqueued(worker: StatusWorker)
    end

    test "an invalid message returns an error" do
      params = Test.Factory.SmsMessageFactory.params(%{message_sid: ""})
20    assert {:error, _} = Conversations.create_sms_message(params)
      refute_enqueued(worker: StatusWorker)
    end
  end
```

We are able to use the Oban test functions because Oban is configured to manual testing mode (you updated config/text.exs in Add Oban to Our SMS App, on page 161). The assert_enqueued (line 9) and refute_enqueued (lines 15 and 21) functions are used to test whether a particular job is enqueued or not.

The arguments to the testing functions are optional, but they provide scoping to the particular job you're targeting. This is important because you don't want your tests to fail as more job types are introduced to your application.

Test Job Implementation

In the previous section, we covered job creation. But we still need to test that the job works correctly.

Our next test will cover the StatusWorker module. You can take different approaches for testing Oban workers. Oban offers a perform_job/3 helper function, but it's often not needed. A worker is just a module that implements the perform/1 function, so we can call that function directly. We'll use this simple approach for our test.

Create test/phone_app/conversations/worker/status_worker_test.exs and add the following code:

phone_app/test/phone_app/conversations/worker/status_worker_test.exs

```
Line 1  defmodule PhoneApp.Conversations.Worker.StatusWorkerTest do
    use PhoneApp.DataCase, async: true

    alias PhoneApp.Conversations.Worker.StatusWorker
5   alias Test.Factory.SmsMessageFactory

    defp setup_bypass(message, status: status) do
      bypass = Bypass.open()
      Process.put(:twilio_base_url, "http://localhost:#{bypass.port}")
10
      Bypass.expect_once(
        bypass,
        "GET",
        "/Accounts/account_sid/Messages/#{message.message_sid}.json",
```